

Time is Brain, and so is Compute

A technical note on modularising a brain-age prediction workflow with LightGBM, Optuna, and Modal

David Andai MBChB, MSc Neuroscience Candidate | andaidavid@afrogia.com

2026-06-03

Introduction

Neuroimaging research increasingly relies on machine learning to predict clinically relevant outcomes (e.g., dementia, brain age gap) from brain-derived features. Improved predictive models can help validate where a disease process may be reflected in the data. In practice, however, strong benchmark performance does not automatically translate into practical or useful deployment, especially in low- and middle-income settings.

Some of the main constraints are straightforward:

- Access to MRI scan is limited.
- Pre-processing and *analysis pipelines* are costly to run.
- MRI Scans often happen late in the clinical pathway, when a diagnosis is already suspected.
- Clinical presentation is variable across patients.
- Scanner and site effects introduce additional noise.

For this experiment, the focus is on the analysis pipeline rather than the novelty of the model. Brain age gap modelling already has a solid methodological foundation in the literature. Here, I use it as a practical case study to examine the time cost of running neuroimaging workflows and to show how the process can be made easier to run, cheaper to maintain locally, and more reusable.

Project goal [From Jupyter Notebook to Modal]

This experiment modularises a LightGBM-based brain-age pipeline and runs the computationally time-consuming training stage on Modal. The aim is to replace a monolithic notebook workflow with a reproducible remote pipeline that is easier to maintain and extend.

Reason for doing this:

- The code becomes even more reusable.
- We make it easier to try different model architectures.
- We can easily adjust various computational environments.

Specific objectives

- Modularise the LightGBM training workflow
- Reproduce the age-prediction and feature importance output in a structured pipeline
- Prepare the setup for later reuse of trained fold outputs and downstream analysis.

With the objectives now set, it is useful to place this experiment in context. The original paper and accompanying code provide the basis for the workflow (Link: <https://www.nature.com/articles/s41591-026-04302-z>). However, the published code does not include benchmark logs for wall-clock runtime, CPU/GPU execution time, or computational cost.

The main note is that computational time increases with the number of features and the number of Optuna trials. When the pipeline is run without modification, the CPU-based workflow takes about 19 hours. This estimate should be treated with caution because it is not independently verifiable from the project artefacts and may vary depending on the CPU used.

The results are shown below. For this reason, runtime reduction will be treated as a motivation and expected benefit, rather than a measured outcome at this stage. The task is therefore to preserve the core modelling features while restructuring the workflow to reduce the local computational burden and make the pipeline easier to run, maintain and extend.

Link to OSF repo: <https://osf.io/yagdh/files/osfstorage>

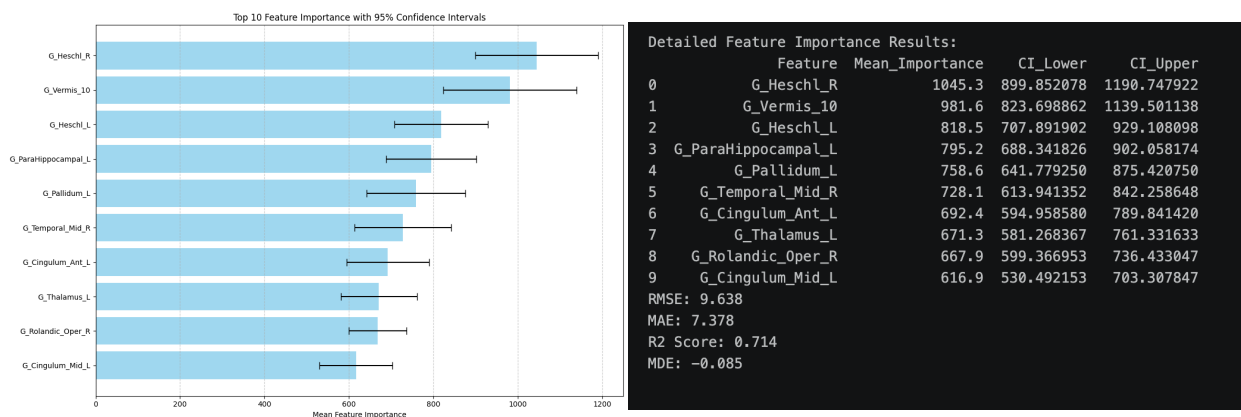


Figure 1: Top 10 Feature Importance and Model Metrics from Jupyter Notebook

(just saved you 19 hrs run time)

My implementation

The code is available on Github (https://github.com/Dr-andai/modal_lgm.git)

- data_utils.py: loads the dataframe, drops rows without usable age labels and prepares features.
- config.py: defines excluded columns, the hyperparameter search space and cross-validation settings.
- models_utils.py: defines the Optuna objective and final LightGBM training routine.
- train_fold.py: runs one outer-fold training job with inner cross-validation and hyperparameter tuning.
- main.py: orchestrates remote execution on Modal, stores fold outputs, and aggregates final predictions.
- feature_importance.py: builds a consolidated feature-importance table and plot from saved fold artefacts.

Experiment setup

- Dataset file: df.csv, rows 12,117
- Feature count used by the model: 116, target variable: Age. Derived output: BAG = Prediction - Age
- Outer cross-validation folds: 10, Inner cross-validation folds: 10. Optuna trials per outer fold: 100

Results

Table: Comparison of notebook and Modal model performance metrics

Metric	Notebook	Modal	Absolute change	Relative change
RMSE	9.638	9.970	+0.332	+3.4%
MAE	7.378	7.638	+0.260	+3.5%
R2	0.714	0.694	-0.020	-2.8%
Mean error	-0.085	-0.367	-0.282	more negative bias

The results below show the runtime on Modal. One is where I maxed out the trials at $n=100$ and 10 T4 GPUs, got an error after a 1-hour timeout, then ran the exact same trials with Optuna ($n=100$, 12-hour timeout).

With what appears to be a reduction in time cost to about 4 hours, a GPU cost of about \$8 was incurred, cushioned by Modal's credits. I was able to get close to the notebook's values. However, there's a noticeable difference in the confidence interval margins for feature importance, even though I had comparable model outputs. This requires further investigation.

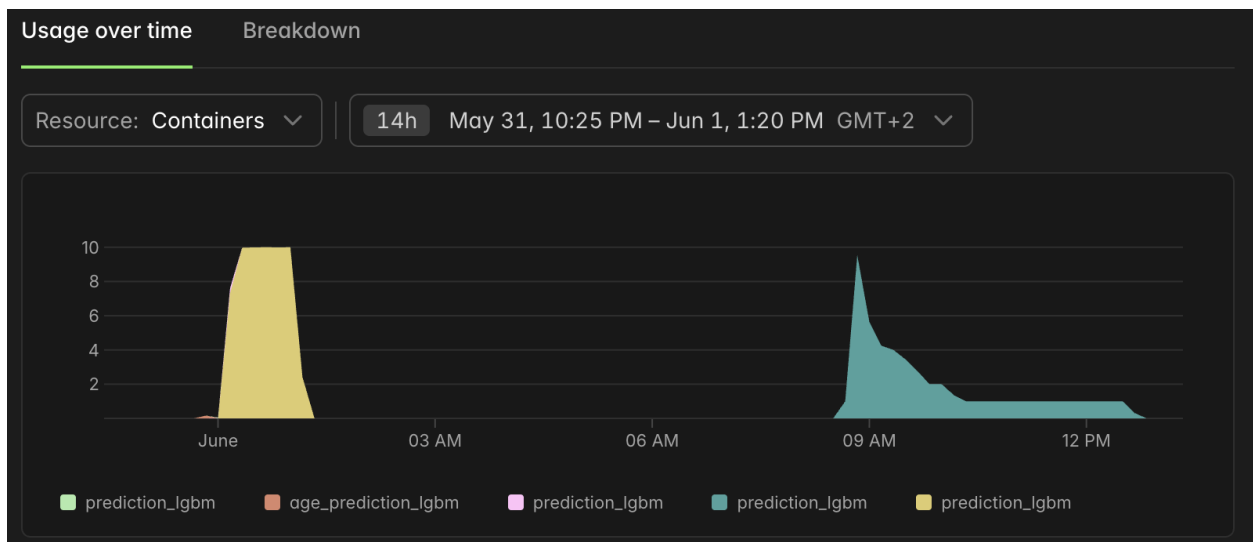


Figure 2: Modal experiment run time

Prediction summary from predictions.csv:

- Mean chronological age: 59.63
- Mean predicted age: 59.26
- Mean brain-age gap: -0.37
- Brain-age gap standard deviation: 9.96

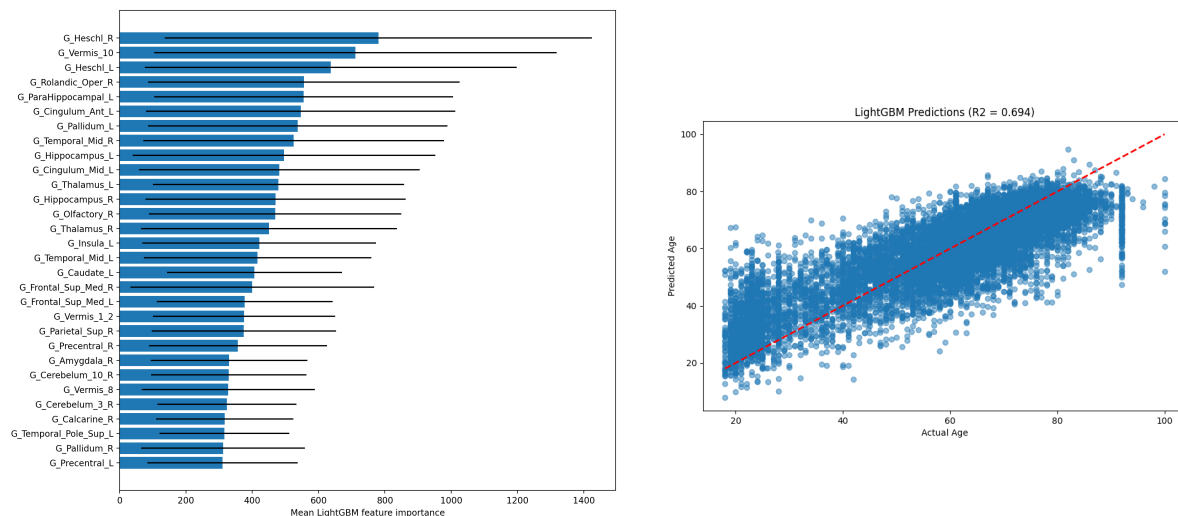


Figure 3: Top 30 Feature Importance and Predicted age vs actual age

Discussion

In this experiment, using accessible GPUs reduced the runtime to about 4 hours. This provides a useful starting point: the workflow can now be modularised while still reproducing the age-prediction outputs and feature-importance results.

However, GPU access is not without trade-offs. Cost must be considered, especially when changing the number of Optuna trials or rerunning the experiment multiple times. This run used a T4 GPU, and performance may differ on other hardware, such as an A100 or Apple MPS. This makes runtime, cost, and hardware choice important considerations in the pipeline design.

For neuroimaging teams, the lesson is that experimental pipelines should not only focus on model performance. They also need to be reusable, affordable to run and easy to adapt across multiple experiments.

At this stage, the pipeline saves fold-level outputs and feature importance, but it does not yet package a single reusable model for deployment. Because the workflow is cross-validated, it produces fold-specific models and outputs rather than a single final production-ready model.

In conclusion, `modal_lgm` is a working modular brain-age prediction pipeline built on LightGBM, Optuna, and Modal. In the saved run, it achieved an R^2 of 0.694, RMSE of 9.97, and MAE of 7.64 across 12,117 usable observations. Its main contribution is the shift from a monolithic notebook to a remote, fold-based, reproducible training workflow.

Areas not covered in this experiment include:

- Deeper evaluation of RMSE, MAE, and reproducibility.
- Interpretation of anatomical feature importance.
- Detailed discussion of Optuna and hyperparameter tuning.
- Packaging a final model for inference.